SWARMA

# Подписи и аутентификация в Swarma

От А до Я: путь команды, ключи и границы доверия

**24 октября 2026 года**

Для оператора и команды. Проект устройства, для ревью.

Утверждённая схема отделена от ещё не определённых контрактов.

# Содержание


**0 Зачем подписывать** . . . . . 3
0.1 Что здесь принято, а что ещё предстоит . . . . . 3

**1 Кто держит ключи** . . . . . 4

**2 Критическая команда с телефона** . . . . . 5

**3 Обычное сообщение из Element** . . . . . 6
3.1 Почему не подписывать каждую реплику . . . . . 6
3.2 Где заканчивается переходный режим . . . . . 6

**4 Агент подписывает агенту** . . . . . 7

**5 Новое и потерянное устройство** . . . . . 8

**6 Когда проверка отказывает** . . . . . 9

**7 Что находится в конверте** . . . . . 10

**8 Команда через два сервера** . . . . . 11

**9 Уверенность и право действовать** . . . . . 12
9.1 Как проходит проверка гранта . . . . . 12

**10 Как устроить Signer и что решить** . . . . . 13
10.1 Разделение ответственности . . . . . 13
10.2 Открытые решения . . . . . 13

# 0 Зачем подписывать

**Аутентификация** устанавливает отправителя. **Авторизация** отвечает, разрешено ли ему это действие. Подпись связывает байты с ключом; грант ограничивает действие, цель и условия. Шифрование скрывает содержимое от посторонних, но само по себе права не выдаёт.

1. При **A0** имя отправителя утверждает homeserver носителя. Захваченный сервер или учётная запись могут выдать команду от чужого имени. Имя на экране и членство в комнате происхождения команды не доказывают.
2. При **A1** проверено E2EE-устройство человека либо подпись ключа агента. Это сильнее серверного утверждения, но ещё не подтверждение человеком конкретного критического действия.
3. Подписанный конверт связывает автора, содержимое, адресата и комнату. Проверка цепочки и отзывов устанавливает, действует ли ключ. Проверка гранта — можно ли исполнить просьбу.
4. Критическая команда требует подписи ключа устройства с присутствием человека. Человек видит точное действие до Face ID, Touch ID или касания аппаратного ключа. Корень оператора ежедневную переписку не подписывает.

**Пример.** Сообщение «удали черновик» под знакомым именем не разрешает удаление. Даже правильная подпись не поможет, если грант разрешает только чтение. Пересказ агентом «владелец одобрил» остаётся данными.

## 0.1 Что здесь принято, а что ещё предстоит

Swarm Signer, аппаратный корень, аттестации устройств и единое ядро — утверждённая схема. Эта записка не удостоверяет, что весь путь уже реализован. В книге «Swarma» 1.2 мост с воротами и ключник описаны как целевая архитектура; работающий обмен сообщениями не доказывает её готовности.

Основание: книга «Swarma» 1.2, главы «Ключи и ключник», «Гранты и полномочия», «Путь сообщения и мост», «Допуск к носителю», «Идентичность агента»; утверждённая схема Signer от 23–24 октября 2026 года. Новая схема уточняет повседневную подпись; формат ячеек Swarm Chain она не заменяет.

# 1 Кто держит ключи

**Аттестация** связывает публичный ключ с владельцем, разрешённым назначением и сроком. Корень доверия закрепляют доверенным способом: имя в чате не устанавливает корень. Стрелки ниже обозначают удостоверение, а не выдачу закрытого материала.

| Ключ | Кто держит и где | Что подписывает | Как прекращается доверие |
|---|---|---|---|
| Корень оператора | оператор; FIDO-ключ и резерв | аттестации ключей и отзывы; не сообщения | процедура замены или потери корня открыта |
| Ключ устройства Signer | Secure Enclave Mac/iPhone: **P-256/ECDSA**, не Ed25519; FIDO/YubiKey: P-256, у некоторых моделей Ed25519; доступ через key provider | критические команды после присутствия человека | корень подписывает отзыв в реестре; также истечение аттестации |
| Корень ключника и ключ агента | Keymaker: мастер-секрет и выводимые ключи; агентский `swarma-msg` — **Ed25519**; у агента секрета нет | корень удостоверяет ключи; агентский ключ — сообщения и квитанции | отзыв отдельного ключа/личности; при компрометации мастер-секрета — всей ветки |
| Matrix master, self-signing, device | E2EE-клиенты; способ хранения зависит от клиента | цепочку cross-signing; device участвует в E2EE | корневой отзыв доверия к master; смена устройств по правилам носителя |
| OMEMO device | XMPP-клиент; хранение зависит от клиента | обмен OMEMO, не конверт Signer | корневой отзыв аттестации конкретного fingerprint |

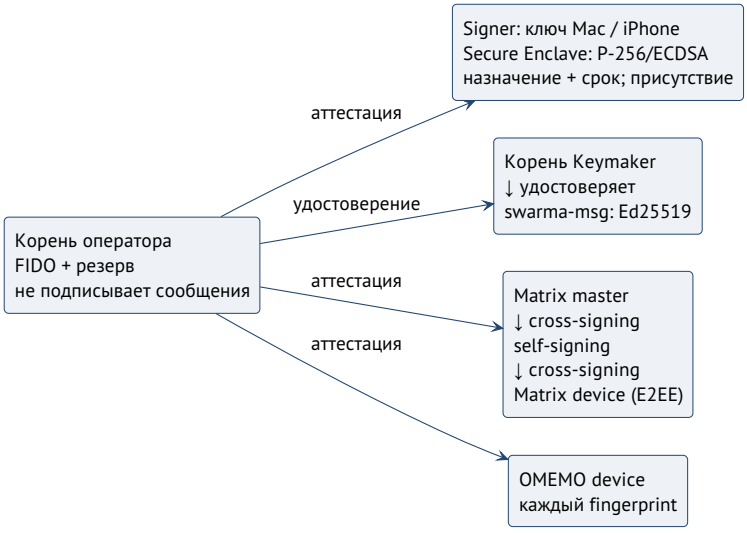



**Рис. 1.1.** От закреплённого корня к ключам разных назначений

Ключник публикует свой корень; владелец подтверждает его. Деривация агентских ключей — отдельный механизм, не деривация из FIDO-корня. Нативные ключи мессенджера не становятся ключами Swarm Signer. Отзыв верхнего звена закрывает доверие к зависимой ветке.

# 2 Критическая команда с телефона

1. Swarm Signer показывает **действие, конкретную цель и адресата**. Содержание готовится целиком: человек не подтверждает неизвестный хэш.
2. Ядро готовит конверт и обращается к key provider платформы. Face ID подтверждает присутствие; ключ **P-256/ECDSA** остаётся в Secure Enclave. Альтернатива — касание FIDO-ключа.
3. Провайдер возвращает подпись. Ядро помещает конверт в содержимое события Matrix, не меняя подписанные данные после подтверждения.
4. Принимающий мост сохраняет событие. Ядро проверки устанавливает подпись, цепочку до закреплённого корня, назначение ключа, срок и отзыв; сверяет тело, комнату, адресата, время и nonce.
5. Ворота доверия Seraph проверяют грант: действие, цель, условия и требуемую уверенность. Никакая подпись не расширяет грант. Исполнитель проверяет основание перед эффектом.
6. Только после допуска действие может исполниться. Иначе — отказ с причиной и аудитом, без эффекта. Квитанция доставки не подтверждает исполнения.

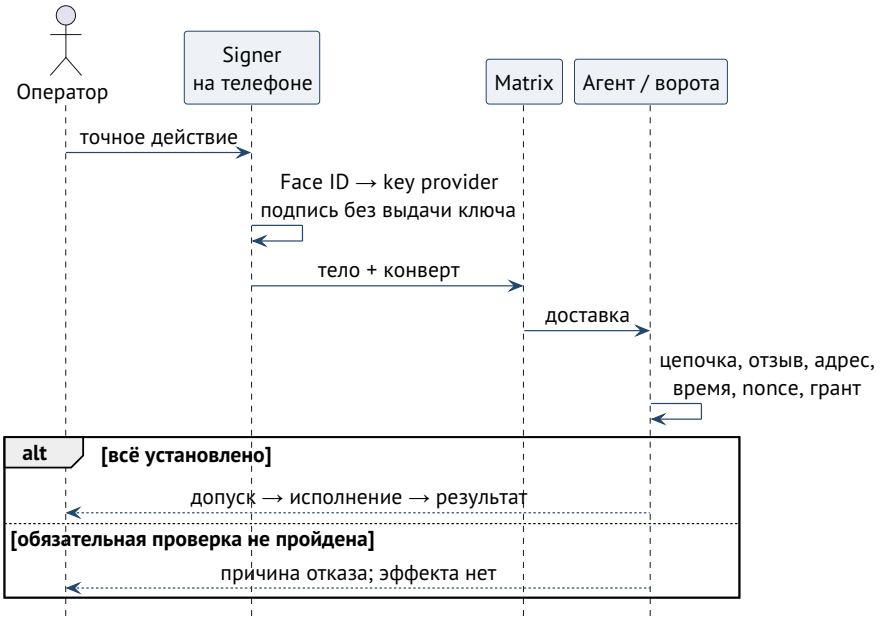



**Рис. 2.1.** Подтверждение точного действия и проверка до эффекта

Расширяющее действие требует **A3** — телефонного подтверждения точного действия. Критичность задаёт грант: корневые операции, release tags, выдача грантов, удаление. Корневую аттестацию или отзыв подписывает FIDO-корень; телефон не получает его секрет.

**Граница.** Наличие поля `sig` не доказывает Face ID. Способ привязать проверяемое свидетельство присутствия к точным байтам ещё должен быть задан контрактом key provider и векторами. При отсутствии требуемого свидетельства критическую команду не допускают.

# 3 Обычное сообщение из Element

1. Оператор пишет в Element с проверенного устройства в E2EE-комнате. Отдельного конверта Signer у сообщения нет.
2. Получатель проверяет E2EE-устройство и цепочку **device ← self-signing ← master**. Master аккаунта должен иметь действующую аттестацию корня оператора; знакомого display name недостаточно.
3. Проверяются актуальность доверия и привязка полной учётной записи отправителя. При выполненных условиях адаптер сообщает **A1**.
4. Сообщение можно использовать как переписку. Оно не подтверждает критическую команду и не выдаёт новый грант. Для такой команды оператор переходит в Signer.

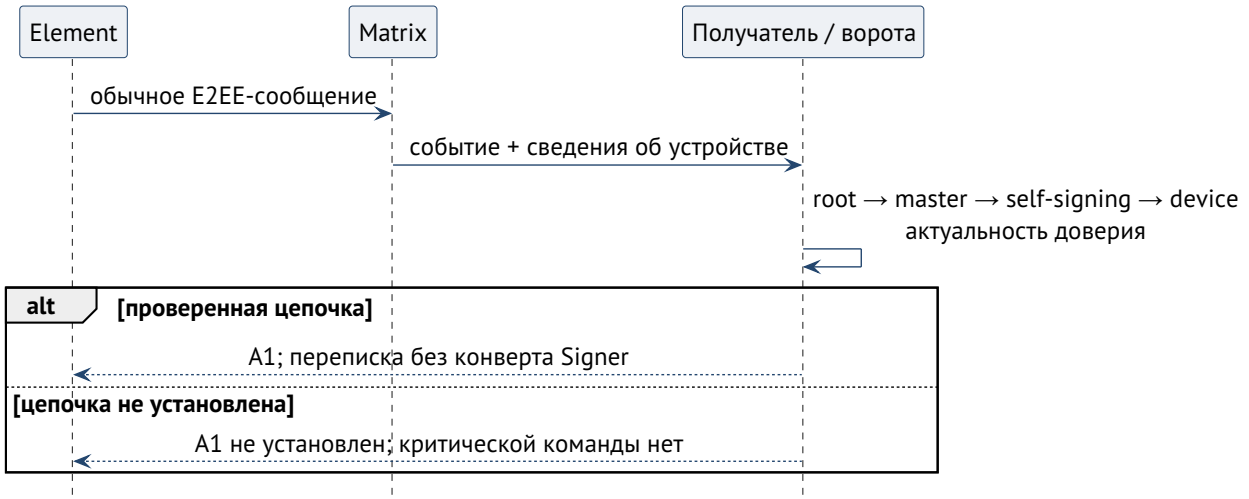



**Рис. 3.1.** E2EE даёт A1 без отдельного конверта Signer

## 3.1 Почему не подписывать каждую реплику

Для обсуждения достаточно установить проверенное устройство собеседника. Нативная цепочка E2EE позволяет не запрашивать биометрию на каждое сообщение. Сервер не может добавить **доверенное** устройство без требуемой цепочки, пока закреплённые ключи и проверяющий клиент не скомпрометированы.

Непроверенное устройство не получает A1 только потому, что аккаунт знаком. Отсутствие проверяемой цепочки не разрешает молча принять критическую команду как A0.

## 3.2 Где заканчивается переходный режим

До ворот и реестра книга допускает команды от заранее названных людей A0/A1 только в их записанных зонах. Зоны задаёт оператор вне области записи агента; их не узнают из чата. После перехода человеческих команд на A2 зоны заменяются грантами, а A0 остаётся данными.

Канал владельца тоже привязывают при подъёме станции: допуск к носителю → провайдер моста → личная комната. Владельца не назначает первый написавший. Привязка комнаты и приглашение не заменяют криптографической проверки и гранта.

# 4 Агент подписывает агенту

1. Агент готовит точные байты для нужной аудитории. Ключ Keymaker для `swarma-msg` — **Ed25519**; этот namespace отделяет сообщения от других подписываемых документов.
2. Агент обращается к Keymaker. Ключник узнаёт UID от ядра ОС и проверяет активную привязку личности к поколению учётки. Имя в тексте запроса личность не выбирает.
3. Ключник подписывает соответствующим агентским ключом и возвращает подпись. Закрытый ключ агенту и мосту не передаётся.
4. CLI `a2a`, как интерфейс общего ядра, отправляет подписанный элемент. Провайдер Matrix перевозит его без переписывания подписанного содержимого.
5. Получатель проверяет подпись, удостоверение ключа до доверенного корня, сроки, отзыв и привязки сообщения. Подтверждённое происхождение соответствует A1 в принятой схеме.
6. Получатель видит **данные коллеги**, а не человеческое распоряжение. Подпись сообщения не даёт `session.approve` и не подтверждает действие от имени оператора.

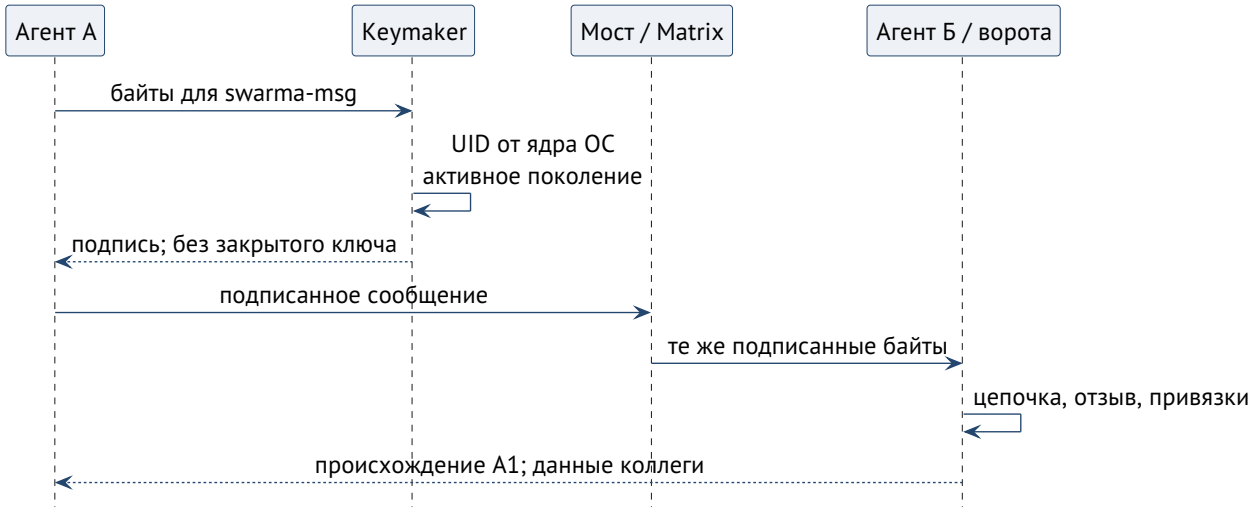



**Рис. 4.1.** Ключник подписывает за установленную ядром ОС личность

**Положительный пример:** подписанный отчёт можно связать с конкретным агентом и проверить неизменность. **Отрицательный:** приписка «оператор разрешил удалить данные» остаётся утверждением автора отчёта. Для разрешения нужно собственное проверяемое основание оператора.

Отдельный типизированный делегированный путь требует своего опубликованного контракта действия и гранта; произвольный подписанный чат его не создаёт. Ключник пока спроектирован, а namespace `swarma-msg` должен появиться в его контракте. Точная доменная метка и байты подписываемого запроса здесь не определены.

# 5 Новое и потерянное устройство

**Включить новое устройство**

1. Secure Enclave Mac/iPhone создаёт **P-256/ECDSA**, а не Ed25519. FIDO-ключ поддерживает P-256 и, в зависимости от модели, Ed25519. Секрет остаётся у платформы/токена за key provider; ядро его не получает.
2. Оператор проверяет ключ и назначение доверенным способом. Корень подписывает аттестацию: **ключ с его алгоритмом, разрешённые действия, период действия**. Вход в приложение аттестацией не становится.
3. Запись публикуется в реестре владельца. Подписанные git-коммиты дают аудит; проверяющий принимает закреплённую подписанную ревизию и проверяемое состояние отзывов.
4. Для Matrix корень удостоверяет cross-signing master аккаунта, а проверка устройства идёт через self-signing. Для XMPP без cross-signing корень удостоверяет каждый OMEMO fingerprint отдельно.
5. Новое устройство допускается только после проверки цепочки и гранта. Конкретная церемония первого закрепления корня и доказательство владения новым ключом ещё требуют контракта.

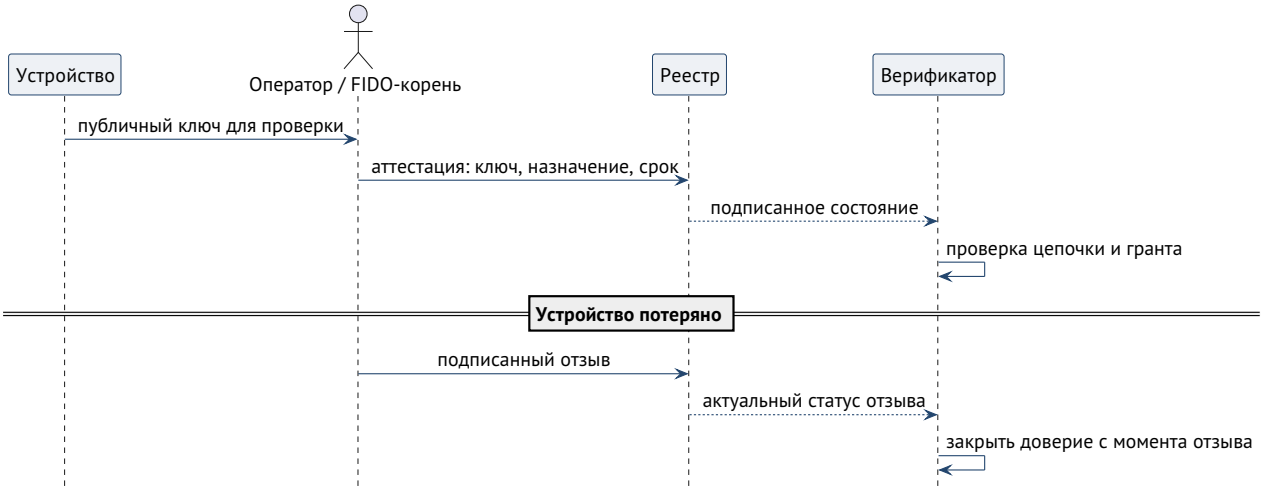



**Рис. 5.1.** Аттестация добавляет доверие, отзыв прекращает его

**Потерять и отозвать**

1. Оператор использует корневой FIDO-ключ или подготовленный резерв, подписывает запись отзыва потерянного ключа и публикует её в реестре.
2. Проверяющие отвергают подписи отозванного ключа **с момента отзыва**. Удалить приложение или выйти из мессенджера для этого недостаточно. У отозванного master теряется доверие ко всей зависимой ветке.
3. Нельзя считать старую копию реестра доказательством отсутствия отзыва. При невозможности проверить актуальный статус новые действия закрыты.
4. Резервирование корня и восстановление при его утрате остаются открытыми. Утраченный корень нельзя «восстановить» голосом из чата. Исторические подписи требуют отдельно определённых временных свидетельств; заявленное самим отправителем старое `ts` не обходит отзыв.

# 6 Когда проверка отказывает

Fail closed: обязательное свидетельство не установлено — нового разрешения и эффекта нет. Ниже — причины для оператора и аудита, **не коды протокола или готовый UI**.

| Условие | Что видит оператор | Что проверить перед повтором |
|---|---|---|
| Реестр нельзя проверить | «Актуальный статус ключа и отзывов не установлен» | доступность, подписанную ревизию и свежесть; не обходить проверку старым снимком |
| Аттестация истекла или ключ отозван | «Ключ не допущен на требуемое время» | новую аттестацию либо другое разрешённое устройство |
| Повтор nonce или `ts` вне окна | «Повтор или недопустимое время сообщения» | доставку, часы, журнал повторов; не исполнять повторно автоматически |
| Другая комната или адресат | «Подписанное назначение не совпадает с доставкой» | полные ID; пересылка не переназначает команду |
| Грант не покрывает действие | «Нет полномочия на это действие и цель» | цепочку, условия и необходимый уровень; подпись прав не добавляет |

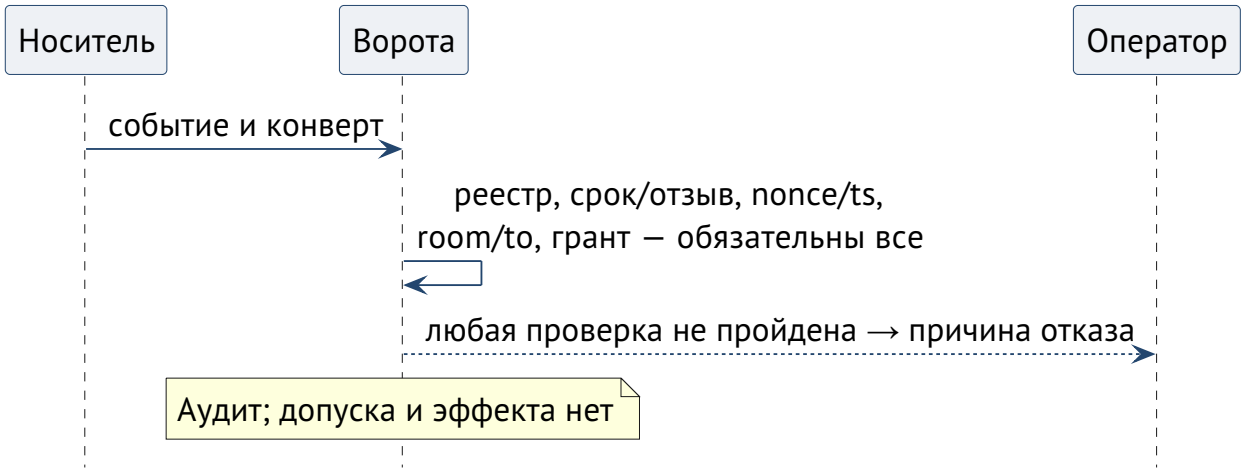



**Рис. 6.1.** Любой обязательный отказ останавливает допуск до эффекта

1. Ворота фиксируют отказ. Неизвестный ключ, неверная подпись и несовпадение хэша тела тоже закрывают допуск.
2. Неверную подпись не обходят доставкой тела как пользовательской команды. Неподписанная переписка остаётся данными, не заменой критической команды.
3. Частичного действия ради проверки нет. Квитанция очереди не подтверждает исполнения.
4. После устранения причины проверяются все условия. При неясном исходе сначала выясняют результат прежнего эффекта: новый nonce не предотвращает повтор внешнего действия.

Fail closed для новых полномочий не отменяет отдельного пути экстренной остановки: в книге остановка не зависит от доступности снимка реестра.

# 7 Что находится в конверте

Формат — **`swarm-envelope/2`**, каноническая форма — **CCJ-1**. В Matrix конверт лежит в содержимом сообщения под именем **`works.relux.swarm.envelope`**.

| Поле | Что связывает проверка |
|---|---|
| `signer`, `key_id` | полную личность отправителя и конкретный удостоверенный ключ |
| `room`, `to` | комнату и полного адресата; не display name или короткий псевдоним |
| `ts`, `nonce` | время в допустимом окне и отсутствие повтора у проверяющего |
| `body_sha256` | SHA-256 точных байтов тела, которое увидел и подтвердил отправитель |
| `sig` | подпись ключа над установленным протоколом представлением |

**Учебный пример, не тестовый вектор.** Тело: `Удалить черновик «пример».` — UTF-8, без завершающего перевода строки. ID вымышлены, хэш вычислен, подпись — заглушка.

```
{
  "works.relux.swarm.envelope": {
    "signer": "@operator:alpha.example",
    "key_id": "phone-example-key",
    "room": "!review:beta.example",
    "to": "@worker:beta.example",
    "ts": "2026-10-24T09:00:00Z",
    "nonce": "example-once-only",
    "body_sha256": "116b69bf4f07138497ff98a33fd51705c4f2645db2fe0daab0fb82d9a9d4b502",
    "sig": "EXAMPLE_NOT_A_SIGNATURE"
  }
}
```

Порядок полей и отступы — для чтения, не канонические байты. Нового поля версии или алгоритма пример не вводит.

1. План **Seraph S0** принимает **Ed25519** и **P-256/ECDSA с low-S**; алгоритм связан с аттестацией. Secure Enclave — P-256, агентский `swarma-msg` — Ed25519.
2. **Подпись: точная ASCII-метка без `0x0a` и управляющих байтов, затем `0x0a`, затем тело CCJ-1.** В S0 — закрытый список попарно различных платформенных меток. Проверяющий сравнивает метку побайтно до первого `0x0a`, без нормализации. Разделитель различает preimage разных меток, поэтому запрет префиксов для безопасности не нужен. Подпись одной роли или домена не принимается как подпись другой.
3. Ядро пересчитывает хэш тела. Получатель строит те же байты; подпись должна защищать назначение, время, nonce и хэш. Изменение тела требует новой подписи.
4. Кодировка подписи, вывод `key_id`, форматы времени и nonce, точный состав подписываемого тела, исключение `sig`, версия и ссылки на гранты ещё открыты. Их определяют контракт и общие векторы Seraph. Формат подписи регистратора сюда не переносится.

# 8 Команда через два сервера

1. Signer подписывает назначение и тело на стороне отправителя. Matrix-сервер `alpha.example` передаёт событие серверу `beta.example` в разрешённую комнату. Это вымышленные имена для примера.
2. Получатель проверяет конверт по закреплённой корневой цепочке, независимо от сервера доставки. Подмена содержимого или назначения не проходит проверку; сервер всё ещё может задержать, скрыть или повторить событие.
3. Затем проверяются актуальный реестр, окно времени, nonce и грант. Сама федерация полномочий не даёт. До прохождения проверок эффекта нет.

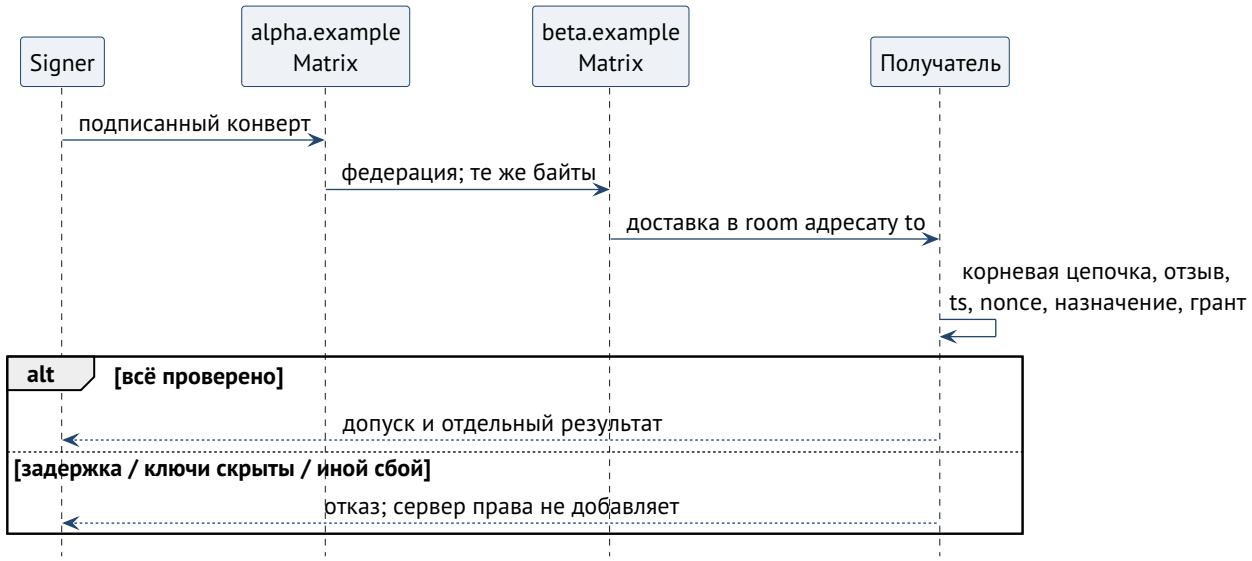



**Рис. 8.1.** Федерация перевозит событие, получатель устанавливает доверие

**A0 удалённого сервера.** Это утверждение чужого homeserver об отправителе, слабее доверия к своему серверу. В целевом режиме A0 — данные и не основание критической команды. Переходное доверие к своему серверу автоматически на чужой не распространяется; исключения потребовали бы явной политики оператора, а не приглашения в комнату.

**Ключи носителя.** Matrix cross-signing и OMEMO device lists приходят через федерацию. Root-аттестация не даёт чужому серверу добавить доверенное устройство. Он может не выдать ключи: цепочку тогда не установить, и допуск закрыт. Это потеря доступности. Сохранность закрытых ключей и корректность клиента остаются допущениями.

**Часы и повторы.** `ts` проверяется относительно окна; nonce хранится каждым верификатором. Задержка федерации окно не расширяет. Размер окна, допустимый сдвиг часов, срок хранения nonce и восстановление после потери журнала ещё открыты. Разные проверяющие не образуют общего хранилища повторов; однократный внешний эффект требует собственного контракта.

**Привязка и правки.** Используются полные `signer` и `to`, ID комнаты. В XMPP полные адреса также нельзя подменять короткими именами; отображение адресов и комнат должен закрепить адаптер. Правка текста требует новой подписи. Redaction может убрать доступные байты, но не даёт нового разрешения; без них проверка невозможна. XMPP — следующий носитель, сроки ещё не определены.

# 9 Уверенность и право действовать

Уверенность описывает свидетельство происхождения. Грант описывает разрешённое действие. Для допуска нужны **оба**, вместе со сроком, отзывом и совпадением цели. Высокий уровень не компенсирует отсутствующий грант.

| Уровень | Свидетельство | Граница действия |
|---|---|---|
| A0 | отправителя утверждает homeserver | в целевом режиме — данные; в переходном — только заранее записанная зона названного человека |
| A1 | проверенное E2EE-устройство человека; в новой схеме также подпись агентского ключа | обычное общение; агентские сообщения остаются данными; критическое действие не разрешено |
| A2 | подпись человеческого ключа над точным конвертом, проверенная воротами по гранту | типизированное действие в пределах гранта; одного A2 недостаточно там, где нужно присутствие |
| A3 | телефон оператора показывает точное действие, человек подтверждает ключом | расширяющие действия: выдача грантов, привязки, допуски; иные критические действия по политике гранта |

## 9.1 Как проходит проверка гранта

1. Найти зарегистрированное действие и конкретную цель; свободный текст не определяет новое полномочие.
2. Пройти цепочку до доверенного корня: подписи издателей, получатели, сроки, условия и отзыв. Каждое дочернее звено только сужает родительское.
3. Проверить требуемую уверенность и присутствие человека. Критичность задаёт грант, в том числе для release tags и удаления. Подтверждение привязано к точному действию, а не ко всему будущему разговору.
4. Повторить проверку на границе эффекта. Детерминированные ворота не спрашивают модель, достаточно ли убедительно сообщение. Дополнительный guard может только сузить допуск.

**Пример:** человеку разрешено читать отчёты; он присылает A3-подтверждение удаления. Уверенность достаточна, полномочия нет — отказ. Обратный случай: грант на удаление есть, но пришла обычная реплика из Element A1 — тоже отказ.

Лестница A0–A3 взята из книги. Новая схема разрешает аппаратное подтверждение также на Mac и FIDO-ключе, но точное отображение этих путей на A3 и формат свидетельства присутствия ещё надо закрепить. До этого их нельзя самовольно объявить эквивалентными телефонному A3.

# 10 Как устроить Signer и что решить

Core выделяется из `a2a-swarm`; CLI `a2a` — один из интерфейсов. Core проверяет и строит конверты, проверяет цепочки и гранты. Адаптеры: сначала Matrix, затем XMPP. Синхронизатор получает снимки реестра; детерминированные ворота выносят вердикт по ним, без сетевого запроса.

Приложения macOS/iOS встраивают core как framework. UI и состояние — Relux Swift, организация — Tuist и packages, вход — device flow. Вход и служебное состояние в Keychain не заменяют root-аттестацию. Секрет остаётся за key provider в Secure Enclave/FIDO; core получает публичные сведения и результат операции.

## 10.1 Разделение ответственности

1. **Seraph:** формат конверта, аттестаций и отзывов; векторы совместимости. Как у регистратора, другие реализации проходят один набор позитивных и негативных векторов.
2. **A2A:** выделение core и интерфейс CLI; одинаковая проверка независимо от транспорта.
3. **Signer:** приложения Mac/iPhone и системный key provider с показом точного действия.
4. **Keymaker:** namespace `swarma-msg` для агентских подписей. Секреты не переезжают в core или процесс агента.

## 10.2 Открытые решения

| Вопрос | Что нужно закрепить до совместимой реализации |
|---|---|
| Язык core | исследование рекомендует Rust/UniFFI с Matrix Rust SDK; ревью ещё не завершено, выбор открыт |
| XMPP | сроки, отображение адресов и комнат, OMEMO-проверка без cross-signing |
| Резерв корня | церемония резерва/восстановления, замена и компрометация FIDO-корня; связь с долгоживущими корнями книги |
| Реестр | доступность, допустимая свежесть и fail closed при недоступности; книга задаёт снимки не старше 60 секунд, но режим Signer ещё требует контракта |
| Присутствие и WC1 | проверяемое связывание присутствия с действием; где живёт witness WC1, кто его проверяет и как учитывается время отзыва |
| Кодировки | кодировка подписи и вывод `key_id`; алгоритмы Ed25519 и P-256 low-S уже предусмотрены планом S0 |
| Байты и повторы | состав подписываемого тела CCJ-1, ссылки на гранты, окно `ts`, хранение nonce, восстановление и внешняя идемпотентность |

Численное окно конверта, коды отказов и формат WC1 здесь не вводятся. Для выпуска нужны опубликованные контракты и общие векторы; принятая архитектура ещё не свидетельствует об их реализации.